Project 1 — Transformations, Projection & Curves from Paintings

CS405 Introduction to Computer Graphics · Fall 2026 · **Mustafa Bozyel** · Student ID: 32417
Repository: github.com/mufasa-349/Computer-Graphs-Project-1 · tag `project1-final`

1 · The painting

Şeker Ahmed Paşa — *Kavunlu Natürmort* (Still Life with Melon). I chose it because it has strong organic contours: the long, smooth upper outline of the melon is a natural target for a piecewise cubic curve, and the cut melon suggests a solid of revolution, which I used for the bonus. The still life also suggests the 3D content: a table standing in front of the painting with a plate, a melon and a small mechanism on it.



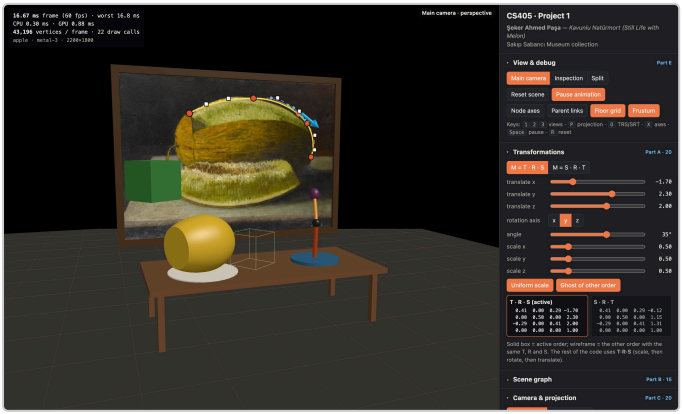
Şeker Ahmed Paşa, *Kavunlu Natürmort*, Sakıp Sabancı Museum.

The application is one WebGPU page (`npm install && npm run dev`). The painting is an sRGB texture on a 4.0 × 2.79 quad at $z = 0$. Everything else derives from $p_{clip} = P \cdot V \cdot M \cdot p_{object}$, which is literally the line `o.clip = cam.viewProj * (obj.model * vec4f(pos, 1))` in the vertex shader. Code is split into `src/math`, `src/scene`, `src/render` and `src/ui`; 31 unit tests (Vitest) cover the matrix, curve and scene-graph code.

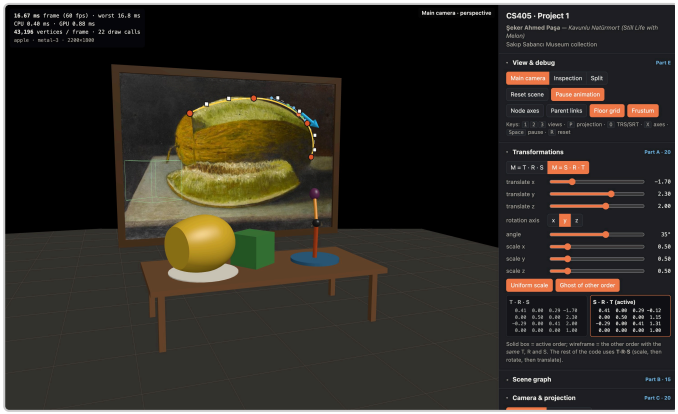
Conventions. Matrices are column-major `Float32Array(16)`, element (r, c) at `m[4c + r]`, uploaded to uniform buffers unchanged; vectors are columns, so `A·B` applies B first. The world is right-handed with +y up, and the camera looks down −z.

2 · Part A — Transformations

`mat4.js` implements identity, multiply, translate, rotate x/y/z, scale, transpose, the rigid inverse $[R\ t;\ 0\ 1]^{-1} = [R^T\ -R^Tt;\ 0\ 1]$, plus a general inverse (for mouse rays and the frustum) and the normal matrix $(M^{-1})^T$. The objects in front of the painting (table, legs, plate, turntable, arm, the swept melon and the green box) are all placed with these matrices. The green box is driven from the UI (translation, axis + angle, scale), with a switch between the two orders and both matrices printed on screen.



M = T·R·S active (solid box, upper left); wireframe = S·R·T.



M = S·R·T active: the solid box moves onto the table.

```
T·R·S   t=(−1.7, 2.3, 2.0), R=R_y(35°), S=0.5
 0.410  0.000  0.287  −1.700
 0.000  0.500  0.000   2.300
−0.287  0.000  0.410   2.000
 0.000  0.000  0.000   1.000
```

```
S·R·T   same T, R, S
 0.410  0.000  0.287  −0.123
 0.000  0.500  0.000   1.150
−0.287  0.000  0.410   1.307
 0.000  0.000  0.000   1.000
```

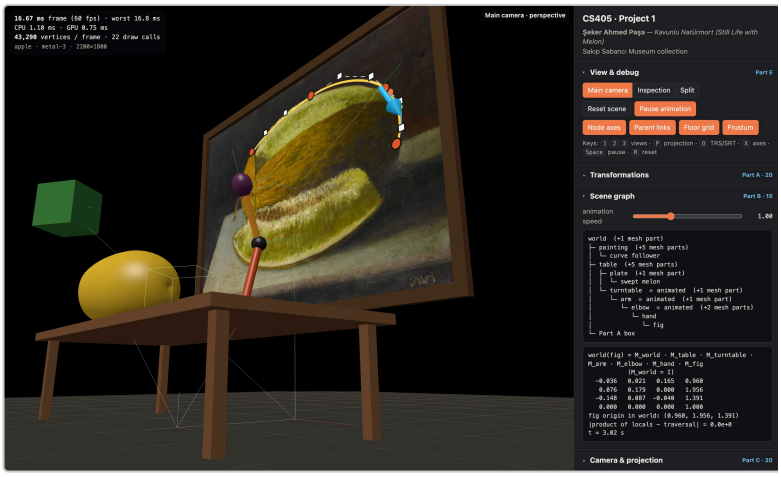
Why they differ. Matrix products do not commute. In **T·R·S** the point is scaled in the box's own axes, rotated about the box's own origin, and only then translated, so the last column is exactly the translation t that I asked for. In **S·R·T** the translation is applied *first*; the rotation then swings the already-moved box around the *world* origin and the scale shrinks its distance from the origin. The last column becomes $S{\cdot}R{\cdot}t = 0.5 \cdot R_y(35°)(-1.7, 2.3, 2.0) = (-0.123, 1.150,$

1.307), so the box lands on the table. The 3×3 blocks are equal here only because a uniform scale commutes with a rotation. With a non-uniform scale (untick *Uniform scale*) S·R ≠ R·S and the S·R·T box is also sheared. **The rest of the code uses T·R·S**: it is the only order in which "size, orientation, position" can be edited independently. It is also what a scene-graph local transform means: placing a child relative to its parent.

3 · Part B — Scene graph

`Node` holds `local`, `parent`, `children` and `world`. A depth-first traversal computes `world = parent.world · local`. The tree has 27 nodes and is six levels deep (world → table → turntable → arm → elbow → hand → fig). Mesh-only children (table top, legs, frame bars, the arm's cylinder) hang under transform nodes so that scaling a shape never scales its children. Three joints are animated: turntable $R_y(0.6t)$, arm $R_z(0.45 \sin 1.3t)$, elbow $R_z(0.9 \sin(2.1t + 1))$. The fig therefore traces a 3D path driven by the whole chain. Debug mode draws every node's local axes (x red, y green, z blue) and a line from each parent to each child.

```
world
├─ painting  T(0,2.544,0)
│  └─ curve follower
├─ floor
├─ table  T(0,0,1.35)
│  ├─ plate  T(−0.95,0.88,0.1)
│  │  └─ swept melon
│  └─ turntable  T(0.95,0.88,0)·R_y(0.6t) ⟳
│     └─ arm  T(0,0.05,0)·R_z(θ₁) ⟳
│        └─ elbow  T(0,0.6,0)·R_z(θ₂) ⟳
│           └─ hand  T(0,0.44,0)
│              └─ fig  T(0,0.07,0)·S(.17,.2,.17)
└─ Part A box  T·R·S from the UI
(mesh-only children omitted)
```



Debug axes and parent links. The panel shows the tree, the fig's world matrix and its check against the explicit product of the local matrices (difference 0).

Leaf world matrix, verified by hand (t = 0)

At t = 0: $R_y(0) = I$, $\theta_1 = 0.45 \sin 0 = 0$ and $\theta_2 = 0.9 \sin 1 = 0.75732$ rad ($\sin\theta_2 = 0.68698$, $\cos\theta_2 = 0.72668$). Every factor above the elbow is a pure translation, so they add: (0, 0, 1.35) + (0.95, 0.88, 0) + (0, 0.05, 0) + (0, 0.6, 0) = elbow at **(0.95, 1.53, 1.35)**. Below the elbow, $R_z(\theta_2)$ acts on the hand and fig offsets (0, 0.44, 0) + (0, 0.07, 0) = (0, 0.51, 0), which gives (−0.51 $\sin\theta_2$, 0.51 $\cos\theta_2$, 0) = (−0.35036, 0.37061, 0). The upper 3×3 is $R_z(\theta_2)\cdot S(0.17, 0.2, 0.17)$:

```
world(fig) by hand
 0.17cosθ₂  −0.2sinθ₂  0     0.95 − 0.35036
 0.17sinθ₂   0.2cosθ₂  0     1.53 + 0.37061
 0           0         0.17  1.35
 0           0         0     1
= [0.1235 −0.1374 0 0.5996; 0.1168 0.1453 0 1.9006;
   0 0 0.17 1.35; 0 0 0 1]
```

```
world(fig) from the traversal (program)
   0.1235  −0.1374   0.0000   0.5996
   0.1168   0.1453   0.0000   1.9006
   0.0000   0.0000   0.1700   1.3500
   0.0000   0.0000   0.0000   1.0000
→ identical to 4 decimals; the panel also prints
  |Π locals − traversal| = 0 at every frame.
```

4 · Part C — Camera and projection

View matrix from two cross products

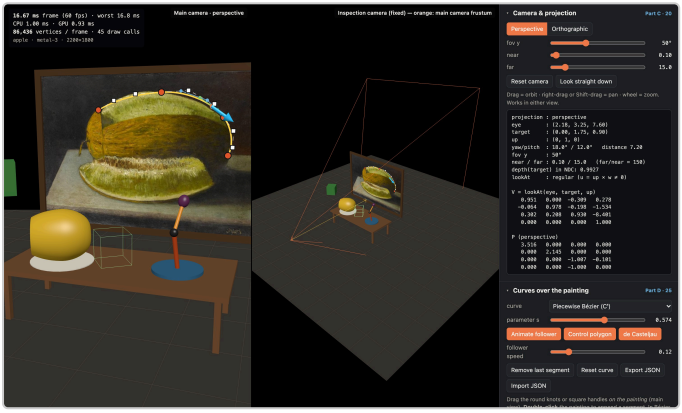
The camera looks from **e** at **a**. Because it looks down its own −z axis, the backward axis is **w** = (**e** − **a**)/|**e** − **a**|. Cross product 1 gives a right vector perpendicular to both up and **w**: **u** = (**up** × **w**)/|**up** × **w**|. Cross product 2 gives the true up vector: **v** = **w** × **u**, which is already unit length because **w** ⊥ **u**. The camera-to-world matrix is C = [**u** **v** **w** **e**; 0 0 0 1], a rigid transform. Its inverse is therefore R^T with $-R^T\mathbf{e}$:

$$V = C^{-1} = [\, u_x\ u_y\ u_z\ -u\cdot e\ ;\ v_x\ v_y\ v_z\ -v\cdot e\ ;\ w_x\ w_y\ w_z\ -w\cdot e\ ;\ 0\ 0\ 0\ 1\ ]$$

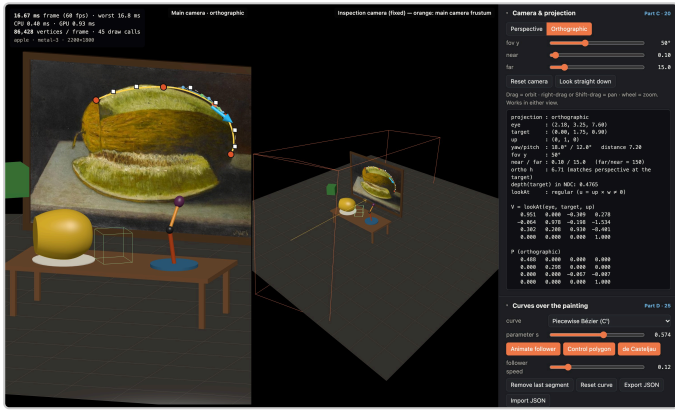
Check: V**e** = 0, and V**a** = (0, 0, −|**e** − **a**|), so the target is on the −z axis (unit-tested). **Degenerate case:** if **up** ∥ **w**, then |**up** × **w**| ≈ 0 and **u** would be NaN. When the norm is below 10^{-6}, `lookAt` substitutes the first world axis (−z, then x, then y) with |axis·**w**| < 0.9. The orbit camera can reach pitch = ±90° exactly ("Look straight down"). The readout then reports that the fallback was used, and the image stays upright with the painting at the top of the screen.

Perspective and orthographic for WebGPU ($z \in [0, 1]$)

With f = cot(fov/2): $x' = (f/\text{aspect})\cdot x$, $y' = f\cdot y$, $w' = -z$, and $z' = A\cdot z + B$. WebGPU needs depth 0 at the near plane and 1 at the far plane (OpenGL uses −1 and 1): $(-An + B)/n = 0 \Rightarrow B = An$, and $(-Af + B)/f = 1 \Rightarrow A = f_{far}/(n - f_{far})$, $B = n\cdot f_{far}/(n - f_{far})$. The orthographic matrix uses the same idea without the divide: $z' = z/(n - f) + n/(n - f)$, so $-n \mapsto 0$ and $-f \mapsto 1$, and x, y map the box [l, r] × [b, t] to [−1, 1]. Its height is $2\cdot d\cdot\tan(\text{fov}/2)$, so switching projections keeps the target at the same size on screen. Default values: fov 50°, near 0.1, far 15. These give the matrices in the panel, e.g. $P_{33} = 15/(0.1 - 15) = -1.007$ and $P_{34} = -0.101$.



Split view: main camera (left), fixed inspection camera (right) with the main frustum in orange.



Same scene, orthographic: the frustum becomes a box and parallel edges stay parallel.

Near/far and depth precision. For perspective, depth is $z_{ndc}(d) = f/(f - n)\cdot(1 - n/d)$, so $dz/dd = f\cdot n / ((f - n)\cdot d^2)$: resolution falls with d^2 and is set mostly by *n*. With a 24-bit depth buffer (step $\approx 6\cdot10^{-8}$), the smallest distinguishable depth difference is:

near / far	Δd at d = 7.2 (target)	Δd at d = 15	z_{ndc} of target
0.1 / 15 (chosen)	$3.1\cdot10^{-5}$	$1.3\cdot10^{-4}$	0.9927
0.01 / 15	$3.1\cdot10^{-4}$	$1.4\cdot10^{-3}$	0.9993
0.1 / 1000	$3.1\cdot10^{-5}$	$1.4\cdot10^{-4}$	0.9862

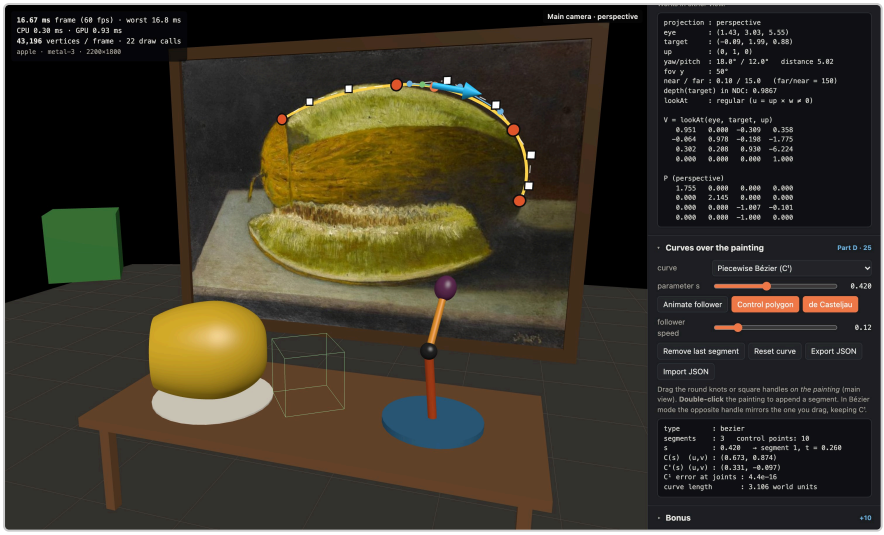
Moving *near* from 0.1 to 0.01 makes precision 10× worse everywhere. Pushing *far* to 1000 barely changes it, but it wastes range and makes the frustum useless as a drawing. Even with my values, 99 % of the depth range is used before the target. I chose 0.1 / 15: the orbit camera never gets closer than 0.6 to its target, and at the default distances the whole scene lies within 15 of the eye. The decorations drawn on the painting sit 0.012–0.032 in front of it, more than 100× the resolution, so they never z-fight. Orthographic depth is linear, with a constant step of $6\cdot10^{-8}\cdot(f - n) \approx 9\cdot10^{-7}$.

5 · Part D — Curves over the painting

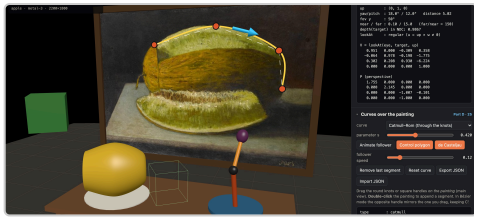
Bézier basis. $B(t) = \Sigma\, B_i(t)\, P_i$ with $B_0 = (1-t)^3$, $B_1 = 3(1-t)^2t$, $B_2 = 3(1-t)t^2$, $B_3 = t^3$ ($\Sigma\, B_i = ((1-t)+t)^3 = 1$). **End points:** at t = 0, B_1, B_2 and B_3 all contain a factor t, so they vanish, and $B_0(0) = 1$. Hence $B(0) = P_0$. At t = 1, B_0, B_1 and B_2 contain (1 − t), and $B_3(1) = 1$, so $B(1) = P_3$. ∎ **Tangent:** $B'(t) = 3[(1-t)^2(P_1 - P_0) + 2(1-t)t(P_2 - P_1) + t^2(P_3 - P_2)]$, so $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$. **de Casteljau:** repeatedly lerp neighbouring points by t (4 → 3 → 2 → 1 points). The last point is B(t), and the two level-2 points span the tangent there. The app draws all levels for the segment containing the chosen parameter (blue, green and orange in the figure) and unit-tests that they agree with the Bernstein form.

C¹ joint. Consecutive segments share the knot K, so C^0 holds automatically. With handles H_- (before) and H_+ (after), and equal parameter length per segment, C^1 requires $B'_{left}(1) = B'_{right}(0)$: $3(K - H_-) = 3(H_+ - K)$, i.e. $\mathbf{H_+ = 2K - H_-}$. The handles are opposite and equally long. `enforceC1` applies this after every edit and mirrors whichever handle the user did not drag; dragging a knot carries both handles. The panel reports the largest tangent jump at the joints, which is $4\cdot10^{-16}$. A global parameter $s \in [0, 1]$ maps to segment $\lfloor sN \rfloor$ with local $t = sN - \lfloor sN \rfloor$.

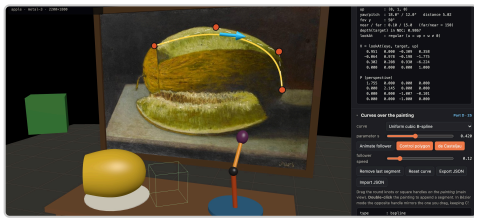
Second family. I implemented both. **Catmull–Rom** interpolates the knots with tangent $(P_{i+1} - P_{i-1})/2$ (C^1). The **uniform cubic B-spline** evaluates its basis $((1-t)^3,\ 3t^3-6t^2+4,\ -3t^3+3t^2+3t+1,\ t^3)/6$ and is C^2 but only approximates the knots. Its end points are repeated three times so it starts and ends on them.



Traced upper outline of the melon: 3 C^1 Bézier segments, control polygon, de Casteljau levels at s = 0.42, and the arrow following the tangent.



Catmull–Rom through the knots.



B-spline: smoother, stays inside the knots.

Interaction and animation. Clicking turns the pixel into a ray: NDC (x, y, 0) and (x, y, 1) are unprojected with $(P{\cdot}V)^{-1}$, moved into the painting's frame with the rigid inverse of its world matrix, and intersected with z = 0. That gives normalised (u, v). Knots and handles are dragged in the painting's plane, and a double-click appends a segment that keeps C^1. The arrow moves at constant speed using an arc-length table (400 chords, binary search). Its frame is x = normalised tangent, z = painting normal, y = z × x. The curve is exported as JSON in normalised coordinates (Appendix).

6 · Part E — Integration and performance

One application with controls to switch between main, inspection and split views, a *Reset scene* button, pause, and toggles for every debug drawing. Keyboard shortcuts and controls are listed in the README. The HUD shows average and worst frame time, CPU time, the GPU render-pass time (timestamp queries) and the number of vertices submitted. Rendering uses 4× MSAA. All objects share one uniform buffer with 256-byte slots and dynamic offsets (one `writeBuffer` per frame). Curves and debug drawings are rebuilt each frame into one vertex buffer.

Measured on my machine: **Apple M1** (7-core GPU, Metal 4), macOS 26.5.2, Chrome 156. Canvas 2200 × 1800 device pixels, 24 HUD samples over 6 s per row:

configuration	frame (avg / worst)	CPU / frame	GPU pass	vertices / frame	draw calls
main view	16.67 / 16.8 ms	0.80 ms	0.91 ms	43,196	22
inspection view	16.67 / 16.8 ms	0.80 ms	0.62 ms	43,240	23
split view (two cameras)	16.67 / 16.8 ms	0.88 ms	0.93 ms	86,436	45
split + all debug drawings	16.67 / 16.8 ms	0.78 ms	0.93 ms	86,624	45

The frame time is capped by the 60 Hz display (vsync), so the useful numbers are CPU ≈ 0.8 ms and GPU ≈ 0.6–0.9 ms, less than 6 % of the frame budget. Split view doubles the vertices but not the GPU time, because the two viewports together cover the same pixels as one full view. The cost is dominated by filling 4× MSAA pixels, not by vertices.

7 · What went wrong and how I found it

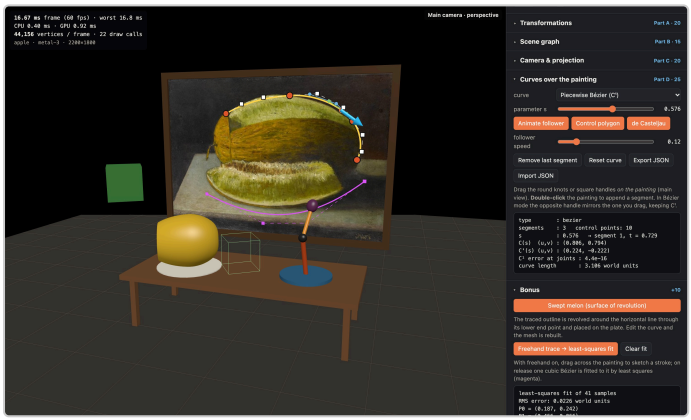
- **Least-squares test failed.** The fitted Bézier for samples of a known curve had an RMS error of $7.8{\cdot}10^{-3}$ instead of ≈ 0. I printed the error for 0–64 iterations (0.092 → 0.025 → 0.0078 → 0.0016) and found the code was correct but the alternation between solving for the points and re-parameterising converges only linearly. I raised the default to 16 iterations and gave the test a tolerance that matches that.
- **The S·R·T box disappeared.** With my first defaults (scale 0.4), S·R·T shrank the translation so much that the box ended up inside the table and the depth test hid it. A screenshot showed this, and it is exactly the effect Part A is about. I chose new defaults where both boxes are visible.
- **Frustum outside the inspection view.** With far = 30, the far plane is ≈ 28 units wide, so the inspection camera only showed lines leaving the screen. Computing that size is also what led to the depth-precision analysis above. I

reduced far to 15 and moved the inspection camera higher and further back.

- **Camera snapped sideways at pitch 90°.** My first fallback took the world axis least aligned with **w** (x). That is valid but rotated the image by 90°. Trying –z first keeps "screen up" pointing into the scene.
- **Normal matrix.** My first hand-written cofactor version was hard to check. I rewrote it as a cofactor table and added a test against transpose(invert(M)) with a non-uniform scale.
- **sRGB.** The canvas format is `bgra8unorm` (not sRGB). Writing shaded colours directly would have looked too dark, so the painting is an `rgba8unorm-srgb` texture (decoded on sampling), lighting happens in linear space, and every fragment shader encodes to sRGB at the end.

8 · Bonus

Surface of revolution. The traced outline (in physical units) is swept around the horizontal line through its lower end point: vertex (x, r cos ϕ, r sin ϕ), 80 profile samples × 56 sectors. Normals come from the profile tangent (dx, dr) → (–dr, dx cos ϕ, dx sin ϕ). The open end gets a flat cap, the melon's cut face. The mesh is placed on the plate and rebuilt whenever the curve changes. **Least-squares Bézier.** Freehand samples d_i get chord-length parameters t_i. With P_0 and P_3 fixed, minimising $\Sigma|B(t_i) - d_i|^2$ gives the 2×2 normal equations $[\Sigma B_1^2\ \Sigma B_1B_2;\ \Sigma B_1B_2\ \Sigma B_2^2][P_1;\ P_2] = [\Sigma B_1 r_i;\ \Sigma B_2 r_i]$, where $r_i = d_i - B_0P_0 - B_3P_3$. These alternate with Newton steps on t_i. A stroke under the melon slice (41 samples) fitted with RMS 0.023 world units (magenta curve below).



Least-squares fit (magenta) of a hand-drawn stroke and the swept melon on the plate.

Appendix — exported curve (`curve.json`)

Piecewise cubic Bézier, 3 segments, C^1. Normalised painting coordinates: u left → right, v bottom → top. Layout K_0 H H K_1 H H K_2 H H K_3.


```
{
  "painting": "Kavunlu Natürmort (Still Life with Melon)",
  "artist": "Şeker Ahmed Paşa",
  "coordinates": "normalised painting coordinates: u ∈ [0,1] left→right, v ∈ [0,1] bottom→top",
  "type": "bezier",
  "piecewiseBezier": {
    "segments": 3,
    "continuity": "C1 (H_after = 2K − H_before at every interior knot)",
    "layout": "K0 H H K1 H H K2 … (knots every third point)",
    "points": [
      { "u": 0.2889, "v": 0.7769 },
      { "u": 0.3639, "v": 0.8367 },
      { "u": 0.4653, "v": 0.8765 },
      { "u": 0.5833, "v": 0.8845 },
      { "u": 0.7014, "v": 0.8924 },
      { "u": 0.8125, "v": 0.8108 },
      { "u": 0.8542, "v": 0.7311 },
      { "u": 0.8958, "v": 0.6514 },
      { "u": 0.8958, "v": 0.5518 },
      { "u": 0.8778, "v": 0.5020 }
    ]
  },
  "knots": [
    { "u": 0.2889, "v": 0.7769 },
    { "u": 0.5833, "v": 0.8845 },
    { "u": 0.8542, "v": 0.7311 },
    { "u": 0.8778, "v": 0.5020 }
  ],
  "leastSquaresFit": null,
  "exportedAt": "2026-11-05T12:00:00.000Z"
}
```